2 实验二 DNS 与 HTTP 协议分析

2.1 实验目的

1）能够分析 DNS 协议的工作过程；

2）分析应用层协议（如 HTTP、HTTPS）的工作过程，理解应用层与传输层及下层协议的关系，分析缓存工作原理及好处。

3）能够依据 HTTP 协议编写简单的 HTTP 客户端程序，从几个网站去读取主页，或者取文件。

2.2 实验内容

1）利用现有校园网络及外网服务器搭建内网、外网环境；

2）用 Wireshark 截取 DNS 报文，分析 DNS 工作过程；

3）用 Wireshark 截获 HTTP/HTTPS 报文，分析报文结构及浏览器和服务器的交互过程；分析 HTTP 协议的缓存机制，分析 HTTPS 的加密传输机制。分析应用层协议跟 TCP/DNS 等协议的交互关系；

4）依据 HTTP 协议，编写简单的 HTTP 客户端程序，从几个网站去读取主页，或者取文件。

2.3 实验原理

2.3.1 DNS 协议概述

DNS（域名系统）协议常被称为“**互联网的电话簿**”。它负责将我们人类容易记住的域名（例如 www.example.com）转换成机器能理解的 IP 地址（例如 192.0.2.1），这样计算机才能找到并连接到目标服务器。

它的工作原理是一个典型的**分布式、层次化**的查询系统。下面为你详细拆解它的工作流程和层次结构。

一、DNS 的层次结构（域名空间树）

DNS 的核心思想是将域名空间组织成一个树状结构。域名由点（.）分隔的多个部分组成，最右边的是顶级域，越往左越具体。

```
根域 (.)
│
├── 顶级域 (Top-Level Domain, TLD) 例如：.com, .org, .cn, .net
│    │
│    └── 二级域 (Second-Level Domain) 例如：example.com, baidu.com
│         │
│         └── 子域 (Subdomain) 例如：www.example.com, mail.example.com
│              │
│              └── 主机 (Host) 例如：特定服务器的主机名
```

1. **根域 (Root Domain)**：用点 . 表示，是 DNS 层级的最顶层。全球有 13 组根域名服务器，由不同机构管理，它们是 DNS 查询的最终起点。

2. **顶级域 (TLD)**：位于根域之下，分为通用顶级域（如 .com, .org）、国家及地区顶级域（如 .cn, .jp）等。

3. **二级域 (Second-Level Domain)**：这是我们在注册域名时可以自定义的部分，例如 example。

4. **子域 (Subdomain)**：是二级域的下级，用于进一步划分不同的服务，例如 www、mail。

二、DNS 协议的工作流程（递归与迭代查询）

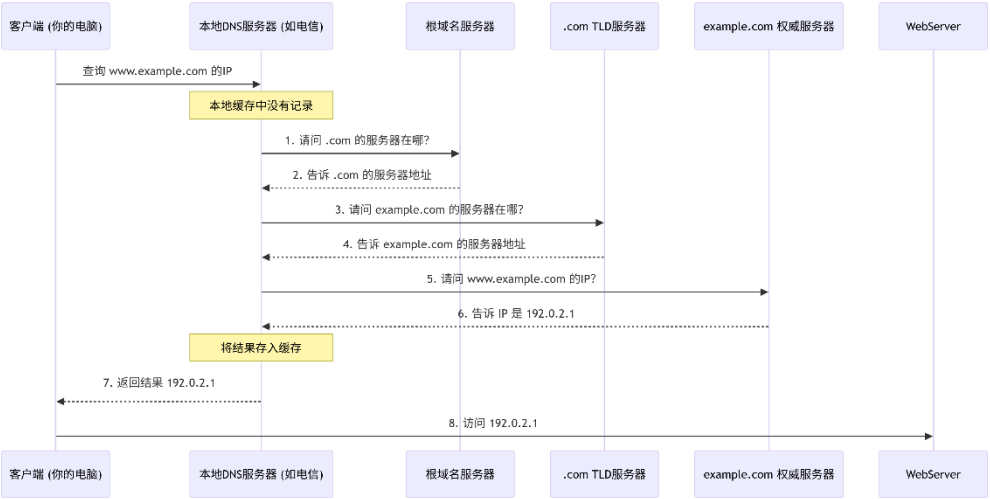
当你在浏览器中输入 www.example.com 并按下回车键时，背后发生了一系列复杂的查询过程。主要涉及两种查询方式：**递归查询** 和 **迭代查询**。

我们可以把参与的角色分为：

- **客户端**：你的电脑（又称“存根解析器”）。
- **本地 DNS 服务器**：通常由你的 ISP 提供商提供，负责代为完成大部分查询工作。
- **权威 DNS 服务器**：负责存储特定域名的最终 IP 地址，是信息的源头。

经典查询流程详解

下以查询 **www.example.com** 为例，说明**通常本地 DNS 服务器会代劳大部分工作**的情况：



详细步骤解释：

1. **客户端发起递归查询：**

你的电脑向**本地 DNS 服务器**发起请求："请告诉我 www.example.com 的 IP 地址，如果你不知道，请帮我去问，然后把最终结果告诉我。" 这个请求是**递归**的，意味着客户端希望服务器完成所有工作并返回最终结果。

2. **本地 DNS 服务器检查缓存：**

本地 DNS 服务器首先查看自己的缓存里有没有 www.example.com 的记录。如果有且未过期，就直接返回给客户端。如果没有，它就要开始工作了。

3. **本地 DNS 服务器进行迭代查询：**

本地 DNS 服务器会代替客户端，去向全球 DNS 系统一层一层地询问。这个过程是**迭代**的：

- **问根域：**它首先问**根域名服务器：**"你知道 www.example.com 的 IP 吗？" 根服务器不知道，但它会告诉它："我不清楚，但 .com 顶级域的权威服务器在那边，你去问它吧。" 并返回 .com 服务器的地址。
- **问顶级域：**本地 DNS 服务器接着问.com **服务器：**"你知道 www.example.com 的 IP 吗？".com 服务器也不知道具体 IP，但它会告诉你："我不清楚，但 example.com 的权威服务器在那边，你去问它吧。" 并返回 example.com 服务器的地址。
- **问权威域：**最后，本地 DNS 服务器问 example.com **的权威服务器：**"你知道 www.example.com 的 IP 吗？" 这个服务器是负责管理 example.com 域下所有记录的，它会在自己的记录中查找，然后返回最终的 IP 地址 192.0.2.1。

4. **返回结果并缓存：**

本地 DNS 服务器拿到了最终的 IP 地址，首先会把它缓存起来，方便下次快速响应。

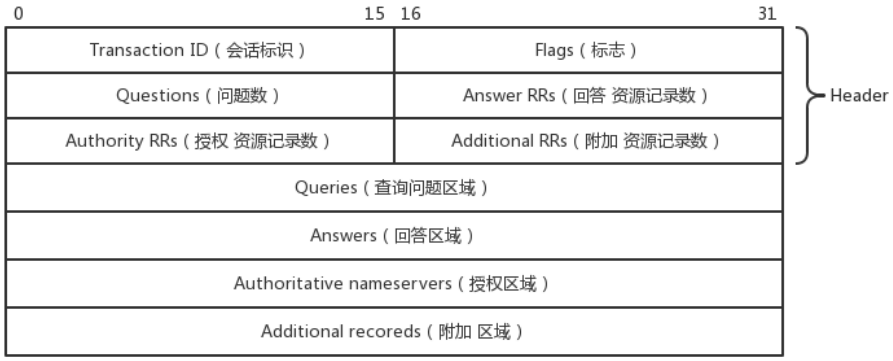
然后，它将结果返回给发起请求的客户端电脑。

5. **客户端建立连接：**

客户端电脑终于拿到了 www.example.com 对应的 IP 地址 192.0.2.1，现在它可以向这个 IP 地址发起 HTTP 请求，最终打开网页。

三、DNS 报文格式及记录类型等

DNS 报文分为两部分：**Header（报头）**和 **Question/Answer（查询问题/回答）部分**。



DNS协议报文格式

1、Header（报头）—— 12 字节固定长度

报头是 DNS 报文的核心，包含了会话 ID、标志位和计数信息。

比特位	字段名	含义与作用
0-15	**Transaction ID**	16 位标识符，由客户端随机生成。服务器在响应时**必须复制该 ID**，这样客户端才能将请求和响应一一对应。
16-23	**Flags**	包含多个控制信息，具体见下表分解。
24-31	**Flags (续)**	标志位剩余部分。
32-47	**Questions**	通常为 1，表示查询部分的问题数量。
48-63	**Answer RRs**	回答部分包含的资源记录数量。
64-79	**Authority RRs**	权威部分包含的资源记录数量。
80-95	**Additional RRs**	附加部分包含的资源记录数量。

Flags 标志位详解（16 位）

比特位	字段名	含义
16	**QR (查询/响应)**	0 = 查询报文；1 = 响应报文
17-20	**Opcode (操作码)**	通常为 0（标准查询）；其他值表示不同操作类型
21	**AA (权威应答)**	响应报文中有效，表示响应的服务器是该域名的权威服务器
22	**TC (截断)**	当报文长度超过 512 字节（UDP 限制）时被置 1，表示报文被截断，需要重新用 TCP 发起查询
23	**RD (期望递归)**	查询报文中置 1，表示希望 DNS 服务器进行递归查询
24	**RA (可用递归)**	响应报文中置 1，表示 DNS 服务器支持递归查询
25-27	**Z (保留)**	保留位，必须为 0
28-31	**Rcode (返回码)**	响应报文中有效：0=无错误；1=格式错误；2=服务器故障；3=域名不存在（NXDOMAIN）

2、Question（问题部分）

这部分告诉 DNS 服务器你要查询什么。

字段	长度	描述
QNAME	**可变**	要查询的域名，采用特定格式编码：每个标签段前面加一个字节的长度，最后以 0 结束。例如 www.example.com 被编码为 3www7example3com0
QTYPE	**2 字节**	查询的记录类型：1=A 记录（IPv4）；28=AAAA（IPv6）；15=MX（邮件交换）；5=CNAME（别名）；2=NS（名称服务器）等
QCLASS	**2 字节**	通常为 1，表示互联网地址（IN）

3、Answer/Authority/Additional（资源记录部分）

这三部分的结构完全相同，都使用**资源记录（RR，Resource Record）**格式。区别在于它们的位置和含义不同：

- **Answer**：回答查询的问题
- **Authority**：指向权威域名服务器
- **Additional**：提供额外的信息（如 A 记录的 IP 地址）

资源记录（RR）格式

字段	长度	描述
NAME (域名)	可变	资源记录对应的域名，可能使用压缩指针
TYPE (类型)	2 字节	同 QTYPE，表示这条记录的类型
CLASS (类)	2 字节	同 QCLASS，通常为 1（IN）
TTL (生存时间)	4 字节	缓存时间，单位秒
RDLENGTH(数据长度)	2 字节	RDATA 字段的长度
RDATA (资源数据)	可变	根据 TYPE 不同而不同：如下记录类型描述。

- **记录类型 (Record Types)**：DNS 不仅仅存储 **A 记录**（IPv4 地址）。常见的还有：
  - **AAAA 记录**：域名对应的 IPv6 地址。
  - **CNAME 记录**：别名记录，将一个域名指向另一个域名。
  - **MX 记录**：邮件交换记录，指定域名的邮件服务器。
  - **NS 记录**：指定该域名由哪个权威 DNS 服务器负责解析。

四、DNS 的缓存

缓存 (Caching)是 DNS 高效运行的关键。查询结果会在各级服务器和客户端本地缓存一段时间（由 TTL 值控制），避免重复查询，大幅减轻根和顶级域的负载。

权威应答与非权威应答：直接从负责该域名的权威服务器返回的结果是**权威应答**。而从缓存中返回的结果是**非权威应答**。

DNS 缓存是分层级的，主要存储在三个地方：

1. 浏览器缓存：当你访问一个网站后，浏览器会把 DNS 解析结果暂时存起来。

- 不同浏览器的缓存时间不同，通常只有几分钟。
- Chrome 浏览器查看缓存：chrome://net-internals/#dns

2. 操作系统缓存，浏览器查询不到时，会向操作系统请求 DNS 结果，操作系统也有自己的 DNS 缓存。

3. 本地 DNS 服务器缓存，如果你的电脑没有缓存，请求会发给 ISP 或公共 DNS 服务器

（如 8.8.8.8）。这些服务器也有强大的缓存，大多数常见域名的查询在这里就被直接回答了，不需要去问根域名服务器。

TTL 是控制缓存的核心参数，它告诉 DNS 客户端："这条记录你可以缓存多久"。

- **TTL 值**：单位是秒，由域名的权威 DNS 服务器设置。
- **常见 TTL**：
  - 稳定的大网站：3600 秒（1 小时）甚至更长
  - CDN 加速的网站：可能只有 60-300 秒，方便快速切换节点
  - 做故障切换的网站：可能设置 30-60 秒，实现快速切换

DNS 缓存的好处

1. **降低延迟**：直接从本地获取，毫秒级响应，不需要网络往返
2. **减少带宽**：避免大量的 DNS 查询流量
3. **减轻根服务器负担**：大部分查询在本地 DNS 服务器就被解决了
4. **提高可靠性**：即使权威 DNS 服务器临时故障，缓存还能继续提供服务

DNS 缓存**带来的问题**

1. **DNS 传播延迟**：当你修改域名的 IP 地址（如更换服务器），全世界生效需要时间，因为各地的缓存需要等到 TTL 过期才会更新。
2. **缓存污染**：如果恶意的 DNS 服务器返回错误的结果并被缓存，会导致用户被引导到钓鱼网站。

2.3.2 HTTP/HTTPS 协议概述[1]

接着深入了解一下 HTTP/HTTPS 协议的工作原理。当你打开浏览器访问一个网站时，背后上演的正是这个协议的工作过程。可以把 HTTP/HTTPS 想象成浏览器和 Web 服务器之间**对话的通用语言**。下面我们就从一次完整的页面访问过程，来拆解它的工作原理。

在介绍流程前，先明确两个关键角色：

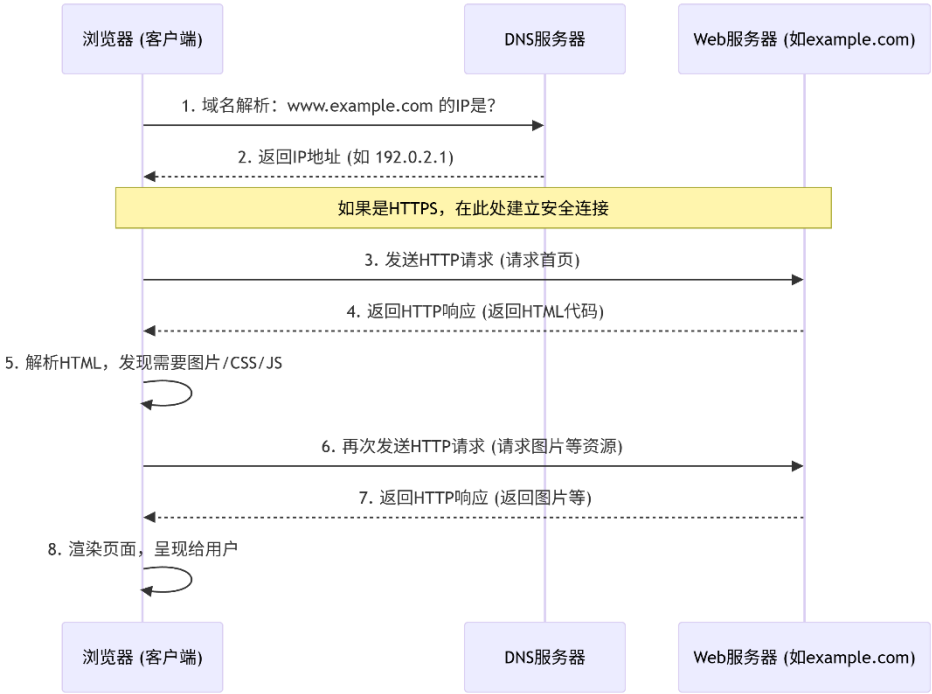
- **客户端**：通常是你的浏览器，它发起请求。

[1]这部分内容参考了 CSDN 博主「有抱负的小狮子」的原创文章，有改动。

- **服务器**：远程的计算机，它存储着网页、图片等资源，并等待客户端的请求，然后返回响应。

一、一次完整的网页访问流程

假设你在浏览器地址栏输入 https://www.example.com 并回车，直到看到页面，背后经历了以下过程：



步骤详解

1. 解析域名 (DNS 查询)

在你输入网址按下回车的那一刻，浏览器做的第一件事就是找到这个网站的 IP 地址。它会向 **DNS 服务器**发起查询，把 www.example.com 这个域名解析成对应的 IP 地址（例如 192.0.2.1）。

2. 建立连接 (TCP 三次握手)

有了 IP 地址，浏览器就会尝试与目标服务器建立连接。这个过程遵循 **TCP/IP 协议**，核心是“三次握手”，目的是确保双方都能正常收发数据，为后续的可靠通信打好基础。

2.5 (可选) TLS 握手 (仅限 HTTPS)

如果你访问的是 https:// 开头的网站，在 TCP 连接建立后，还会有一个 **TLS 握手** 过程。这是 HTTPS 保证安全的关键步骤：

- **身份验证**：浏览器验证服务器出示的 **SSL/TLS 证书**，确认你访问的 www.example.com 确实

是它自己，而不是一个假冒的网站。

- **协商密钥**：双方通过一系列交互，协商出一个只有它们知道的“会话密钥”，用于加密后续的所有通信数据。这一步完成后，浏览器地址栏会出现一把**小锁标志**，意味着从现在开始，所有传输内容都是加密的，即使被截获也无法解读。

3. 发送 HTTP 请求 (Request)

连接建立后，浏览器会按照 **HTTP 协议** 的格式，向服务器发送一个请求。这个请求报文通常包含以下几部分：**请求行**、**请求头**和**请求体**（可能）。详细的参考下面。

4. 服务器处理并返回 HTTP 响应 (Response)

服务器收到请求后，会根据路径 / 找到对应的网页文件（通常是 index.html），然后生成一个 HTTP 响应报文发回给浏览器。这个响应报文也包含三部分：**状态行**、**响应头**和**响应体**。详细的参考下面。

5. 浏览器解析与渲染

浏览器收到 HTML 代码后，会从上到下开始解析。在解析的过程中，它会发现 HTML 代码里还引用了其他资源，比如：

- CSS 文件 (用于样式)
- JavaScript 文件 (用于交互)
- 图片、视频等

6. 获取子资源

每当发现一个外部资源，浏览器就会**重复步骤 3 和 4**，向服务器发起新的 HTTP 请求来获取这些资源。为了提高效率，浏览器通常会同时开启多个连接来并行下载。

7. 最终呈现

当浏览器收集齐了所有的 HTML、CSS 和 JS **文件**后，它会根据这些代码进行布局、绘制，最终把完整的、可交互的网页呈现在你的面前。

总结一下

- **HTTP 是** 一种**无状态**、**明文传输**的协议，定义了客户端和服务器之间通信的格式。
- **HTTPS 是** HTTP 加上 **TLS/SSL 加密层**，它提供了**身份验证**和**数据加密**，保证了传输过程的安全性。
- 一次完整的访问过程，本质上是浏览器和服务器之间通过 **TCP 连接**，按照 HTTP 规则进行多次**请求-响应**的对话，最终获取并渲染出网页。

二、HTTP 协议的历史和发展

超文本传输协议（HyperText Transfer Protocol，HTTP）是一种用于分布式、协作式和超媒体信息系统的应用层协议。HTTP 是 WWW 的数据通信基础。

HTTP 的发展是由蒂姆·伯纳斯-李于 1989 年在欧洲核子研究组织（CERN）所发起。HTTP 的标准制定由万维网协会（World Wide Web Consortium，W3C）和互联网工程任务组（Internet Engineering Task Force，IETF）进行协调，最终发布了一系列的 RFC，其中最著名的是 1999 年 6 月公布的 RFC 2616，定义了 HTTP 协议中现今广泛使用的一个版本——HTTP 1.1。

2014 年 12 月，互联网工程任务组（IETF）的 Hypertext Transfer Protocol Bis（httpbis）工作小组将 HTTP/2 标准提议递交至 IESG 进行讨论，于 2015 年 2 月 17 日被批准。 **HTTP/2** 标准于 2015 年 5 月以 RFC 7540 正式发表，取代 HTTP 1.1 成为 HTTP 的实现标准。

HTTP 是一个客户端终端（用户）和服务器端（网站）请求和应答的标准。客户端通常是浏览器，发起一个 HTTP 请求到服务器上指定端口（默认 80）。客户端称为用户代理（user agent）。应答服务器上存储着一些资源，比如 HTML 文件和图像。称为源服务器（origin server）。在用户代理和源服务器中间可能存在多个“中间层”，比如代理服务器、网关或者隧道（tunnel）。

尽管 TCP/IP 协议是互联网上最流行的应用，HTTP 协议中，并没有规定必须使用它或它支持的层。事实上，HTTP 可以在任何互联网协议上，或其他网络上实现。HTTP 假定其下层协议提供可靠的传输。因此，任何能够提供这种保证的协议都可以被其使用。

通常，HTTP 客户端发起一个请求，创建一个到服务器指定端口（默认 80）的 TCP 连接。HTTP 服务器则在该端口监听客户端的请求。一旦收到请求，服务器会向客户端返回一个状态，比如"HTTP/1.1 200 OK"，以及返回的内容，如请求的文件、错误消息、或者其它信息。HTTP 协议的特点是无状态，就是说 HTTP 协议本身不会对请求和响应之间的通信状态做保存。建立在 HTTP 协议之上的应用如果需要状态，可以使用 cookie 来完成。

三、HTTP 协议的请求和应答

HTTP 协议由 HTTP 请求和 HTTP 响应组成，当在浏览器中输入网址访问某个网站（或点击链接）时， 你的浏览器会将你的请求封装成一个 HTTP 请求发送给服务器站点，服务器接收到请求后会组织响应数据封装成一个 HTTP 响应返回给浏览器。

HTTP 协议的请求

HTTP 请求包括请求行、请求头、请求体，HTTP 响应包括响应行、响应头、响应体。

HTTP 请求的例子如下图所示：

请求行必须在 http 请求格式的第一行。请求头从第二行开始，到第一个空行结束。请求头和请求体之间存在一个空行（如图中 Cookie 行和 name 行之间）。请求体是可选的，可以为空。

请求头是后面几行（直到 Cookie 行），

请求行的格式：请求方式 资源路径 协议/版本，例如：POST /chapter17/user.html HTTP/1.1。**请求方式**很多，如 GET，POST，HEAD 等等。

GET 请求将请求参数追加在 URL 后面，很简单，没有请求体。但 URL 长度限制 GET 请求方式数据的大小。一般的 HTTP 请求大多都是 GET。常见 GET 请求:地址栏直接访问、<a href="">、<img src="">等。

POST 请求：请求参数在请求体处，请求数据大小没有限制，只有表单设置为 method="POST"才是 POST 请求，其他大都是 GET 请求。

HEAD 请求：跟 GET 相似，不过服务端接收到 HEAD 请求时只返回响应头，不发送响应内容。所以，如果只需要查看某个页面的状态时，用 HEAD 更高效，因为省去了传输页面内容的时间。

其他还有 DELETE，OPTIONS，PUT，TRACE，CONNECT 请求等。

请求头从第二行开始，通常以键值对 key:value 方式传递数据。例如：

POST /vk/app/rest/ddp/findModelByType HTTP/1.1

User-Agent: Fiddler

Host: 39.108.107.149:8080

Content-Length: 11

name=城市

key 为规范的固定值，value 为 key 对应的取值，通常是一个值，可能是一组。

HTTP 请求报文头属性

常见请求头

Referer：表示这个请求是从哪个 url 跳过来的。通过百度来搜索淘宝网，那么在进入淘宝网的请求报文中，Referer 的值就是 www.baidu.com。如果是直接访问就不会有这个头。常用于防盗链。

Accept：告诉服务端该请求所能支持的响应数据类型，即 MIME 类型。

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8

MIME 格式：大类型/小类型[;参数]

例如:text/html, text/css，text/javascript，image/*等。

If-Modified-Since:浏览器通知服务器，本地缓存的最后变更时间。与另一个响应头组合控制浏览器页面的缓存。

Cookie：客户端的 Cookie 通过这个属性传给服务端。

User-Agent:浏览器通知服务器，客户端浏览器与操作系统相关信息

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/69.0.3497.100 Safari/537.36

Connection:表示客户端与服务连接类型；Keep-Alive 表示持久连接，close 已关闭。

Host:请求的服务器主机名

Content-Length:请求体的长度

POST /vk/app/rest/ddp/iModelServiceImpl/findModelByType HTTP/1.1

User-Agent: Fiddler

Host: 39.108.107.149:8080

Content-Length: 11

name=城市

Content-Type:请求的与实体对应的 MIME 信息。如果是 post 请求,会有这个头,默认值为 application/x-www-form-urlencoded，表示请求体内容使用 url 编码。

Accept-Encoding：浏览器通知服务器，浏览器支持的数据压缩格式。如 GZIP 压缩

Accept-Encoding: gzip, deflate

Accept-Language：浏览器通知服务器，浏览器支持的语言。各国语言(国际化 i18n)

Accept-Language: zh-CN,zh;q=0.9

Cache-Control：指定请求和响应遵循的缓存机制

对缓存进行控制，如一个请求希望响应返回的内容在客户端要被缓存一年，或不希望被缓存就可以通过这个报文头达到目的。

Cache-Control: no-cache

请求体

当请求方式是 POST 时，请求体有请求的参数，格式如下：

username=zhangsan&password=123

HTTP 协议的应答

HTTP 应答（也称响应）也由三部分组成（响应行+响应头+响应体）。

响应行：

①报文协议及版本；

例如：HTTP/1.1 200 OK

②状态码及状态描述；

状态码：由 3 位数字组成，第一个数字定义了响应的类别

1xx：指示信息，表示请求已接收，继续处理

2xx：成功，表示请求已被成功接受，处理。

200 OK：客户端请求成功

204 No Content：无内容。服务器成功处理，但未返回内容。一般用在只是客户端向服务

器发送信息，而服务器不用向客户端返回什么信息的情况。不会刷新页面。

206 Partial Content：服务器已经完成了部分 GET 请求（客户端进行了范围请求）。响应报文中包含 Content-Range 指定范围的实体内容

3xx：重定向

301 Moved Permanently：永久重定向，表示请求的资源已经永久的搬到了其他位置。

302 Found：临时重定向，表示请求的资源临时搬到了其他位置

303 See Other：临时重定向，应使用 GET 定向获取请求资源。303 功能与 302 一样，区别只是 303 明确客户端应该使用 GET 访问

307 Temporary Redirect：临时重定向，和 302 有着相同含义。POST 不会变成 GET

304 Not Modified：表示客户端发送附带条件的请求（GET 方法请求报文中的 IF…）时，条件不满足。返回 304 时，不包含任何响应主体。虽然 304 被划分在 3XX，但和重定向没有关系。

4xx：客户端错误

400 Bad Request：客户端请求有语法错误，服务器无法理解。

401 Unauthorized：请求未经授权，这个状态代码必须和 WWW-Authenticate 报头域一起使用。

403 Forbidden：服务器收到请求，但是拒绝提供服务

404 Not Found：请求资源不存在。比如，输入了错误的 url

415 Unsupported media type：不支持的媒体类型

5xx：服务器端错误，服务器未能实现合法的请求。

500 Internal Server Error：服务器发生不可预期的错误。

503 Server Unavailable：服务器当前不能处理客户端的请求，一段时间后可能恢复正常，

响应头：

③响应头，也是由多个属性组成；也是用键值对 k: v 表示。

服务器通过响应头来控制浏览器的行为，不同的头浏览器操作不同。

常见响应头	描述
Location	指定响应的路径，需要与状态码 302 配合使用，完成跳转。
Content-Type	响应正文的类型（MIME 类型） 取值：text/html;charset=UTF-8
Content-Disposition	通过浏览器以下载方式解析正文 取值：attachment;filename=xx.zip
Set-Cookie	与会话相关技术。服务器向浏览器写入 cookie
Content-Encoding	服务器使用的压缩格式 取值：gzip

Content-length	响应正文的长度
Refresh	定时刷新，格式：秒数;url=路径。url 可省略，默认值为当前页。 取值：3;url=www.itcast.cn //三秒刷新页面到 www.itcast.cn
Server	服务器名称
Last-Modified	服务器通知浏览器，文件的最后修改时间。与 If-Modified-Since 一起使用。
Cache-Control	响应输出到客户端后，服务端通过该报文头属告诉客户端如何控制响应内容的缓存。常见的取值有 private、public、no-cache、max-age，no-store，默认为 private。缓存时间为 31536000 秒（365 天）

响应体：

④响应体，就是服务器发送给浏览器的正文，即我们真正要的内容（如网页，图片等）；

响应体，响应体是服务器回写给客户端的页面正文，浏览器将正文加载到内存，然后解析渲染显示页面内容

四、HTTPS 协议简介

HTTPS(Hypertext Transfer Protocol Secure)，是以安全为目标的 HTTP 通道，在 HTTP 的基础上通过传输加密和身份认证保证了传输过程的安全性，HTTPS 的安全基础是 SSL，HTTPS 存在不同于 HTTP 的默认端口及一个加密/身份验证层（在 HTTP 与 TCP 之间）。

HTTPS 是通过 SSL/TLS 协议提供的安全连接进行的 HTTP 通信，最初，它被用于安全很重要的通信，如在互联网上传输个人信息和电子支付。随后，由于公共无线局域网的普及、PRISM 的大规模窃听以及中间人攻击的风险增加，用 HTTPS 取代单纯的 HTTP 通信越来越多。

与以纯文本形式发送和接收消息的标准 HTTP 不同，HTTPS 使用 SSL/TLS 等协议对服务器进行身份验证、加密通信内容和检测篡改，这样可以防止欺骗、中间人攻击和窃听等攻击，HTTPS 使用 443 作为已知端口号。

HTTPS 原理：

①客户端将它所支持的算法列表和一个用作产生密钥的随机数发送给服务器；

②服务器从算法列表中选择一种加密算法，并将它和一份包含服务器公用密钥的证书发送给客户端；该证书还包含了用于认证目的的服务器标识，服务器同时还提供了一个用作产生密钥的随机数；

③客户端对服务器的证书进行验证（有关验证证书，可以参考数字签名），并抽取服务器的公用密钥；然后，再产生一个称作 pre_master_secret 的随机密码串，并使用服务器的公用密钥对其进行加密（参考非对称加/解密），并将加密后的信息发送给服务器；

④客户端与服务器端根据 pre_master_secret 以及客户端与服务器的随机数值独立计算出加

密和 MAC 密钥（参考 DH 密钥交换算法）；

⑤客户端将所有握手消息的 MAC 值发送给服务器；

⑥服务器将所有握手消息的 MAC 值发送给客户端。

HTTPS 协议的改进：

①双向的身份认证：

客户端和服务端在传输数据之前，会通过基于 X.509 证书对双方进行身份认证。具体过程如下：

客户端发起 SSL 握手消息给服务端要求连接。

服务端将证书发送给客户端。

客户端检查服务端证书，确认是否由自己信任的证书签发机构签发。如果不是，将是否继续通讯的决定权交给用户选择。如果检查无误或者用户选择继续，则客户端认可服务端的身份。

服务端要求客户端发送证书，并检查是否通过验证。失败则关闭连接，认证成功则从客户端证书中获得客户端的公钥，一般为 1024 位或者 2048 位。到此，服务器客户端双方的身份认证结束，双方确保身份都是真实可靠的。

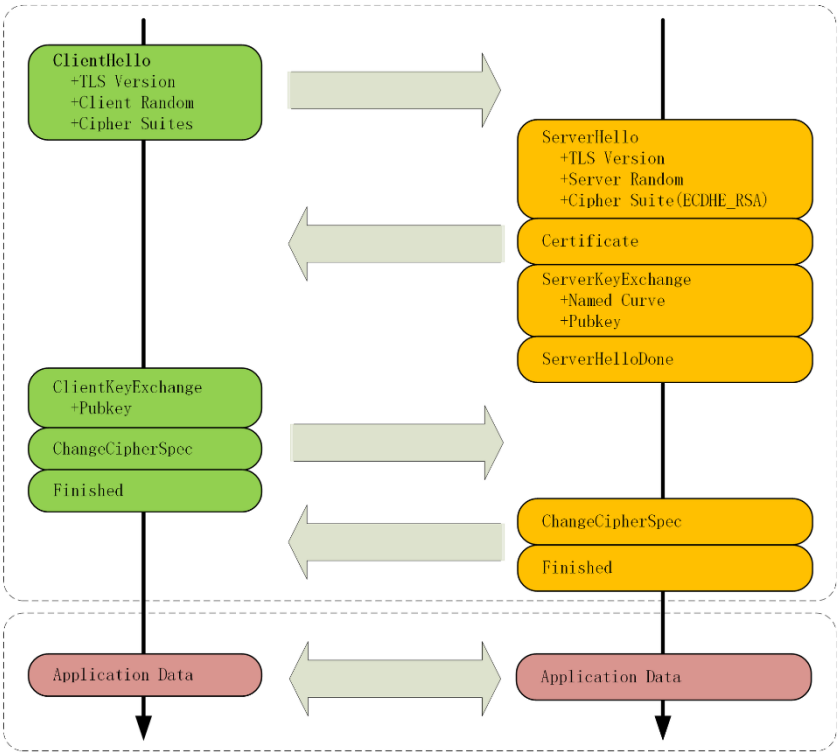
②数据传输的机密性：

客户端和服务端在开始传输数据之前，会协商传输过程需要使用的加密算法。客户端发送协商请求给服务端，其中包含自己支持的非对称加密的密钥交换算法（一般是 RSA）,数据签名摘要算法（一般是 SHA 或者 MD5）,加密传输数据的对称加密算法（一般是 DES），及加密密钥的长度。服务端接收到消息之后，选中安全性最高的算法，并将选中的算法发送给客户端，完成协商。客户端生成随机的字符串，通过协商好的非对称加密算法，使用服务端的公钥对该字符串进行加密，发送给服务端。服务端接收到之后，使用自己的私钥解密得到该字符串。在随后的数据传输当中，使用这个字符串作为密钥进行对称加密。

③防止重放攻击：

SSL 使用序列号来保护通讯方免受报文重放攻击。这个序列号被加密后作为数据包的负载。在整个 SSL 握手中，都有一个唯一的随机数来标记 SSL 握手。这样防止了攻击者嗅探整个登录过程，获取到加密的登录数据之后，不对数据进行解密，而直接重传登录数据包的攻击手法。

可以看到，鉴于电子商务等安全上的需求，HTTPS 对比 HTTP，在安全方面已经取得了极大的增强。总结来说，HTTPS 的改进点在于创造性的使用了非对称加密算法，在不安全的网路上，安全的传输了用来进行对称加密的密钥，综合利用了非对称加密的安全性和对称加密的快速性。



参考资料：深入浅出 HTTPS（https://www.cnblogs.com/huansky/p/13977181.html）。

2.3.3 编写 HTTP 客户端程序

编写一个简单的 HTTP 客户端程序进行连接测试（如连接到 www.xjtu.edu.cn 或 www.hit.edu.cn 服务器 80 端口，获取并打印主页）。

建议使用 Python 编程。使用 Python 编程时，只能使用 socket 模块，**不能使用 Python 的高级 HTTP 模块**（如 requests、urllib.request、http.client 等）。

2.4 实验环境与分组

依托校园网和外网服务器搭建内网、外网环境，每名同学单独完成实验并撰写实验报告。

2.5 实验步骤

2.5.1 DNS 协议分析

步骤 1：在命令窗口执行命令：

执行“ipconfig /displaydns”观察本地缓存；

执行“ipconfig /flushdns”清除本地缓存。

步骤 2：**默认的 DNS 域名解析。**在计算机终端上运行 Wireshark 截获报文，浏览器访问域名（如 www.baidu.com），网站打开后停止报文截获，观察分析 DNS 查询、回复报文分别包含哪些主要内容（UDP 还是 TCP、目的地址与本机默认 DNS 是否相同、源端口和目的端口、域名解析记录类型、解析出的 IP 地址等。DNS 数据包过多时，可设置过滤条件：dns.qry.name contains www.baidu.com）。

DNS 域名解析记录类型：

A 记录：将域名指向一个 IPv4 地址；

CNAME 记录：将域名指向一个域名，实现与被指向域名相同的访问效果，这个域名一般是主机服务商提供的一个域名；

PTR 记录：PTR 记录是 A 记录的逆向记录，又称做 IP 反查记录或指针记录，负责将 IP 反向解析为域名；

AAAA 记录：将主机名（或域名）指向一个 IPv6 地址；

MX 记录：建立电子邮箱服务，将指向邮件服务器地址；

NS 记录：域名解析服务器记录，如果要将子域名指定某个域名服务器来解析，需要设置 NS 记录；

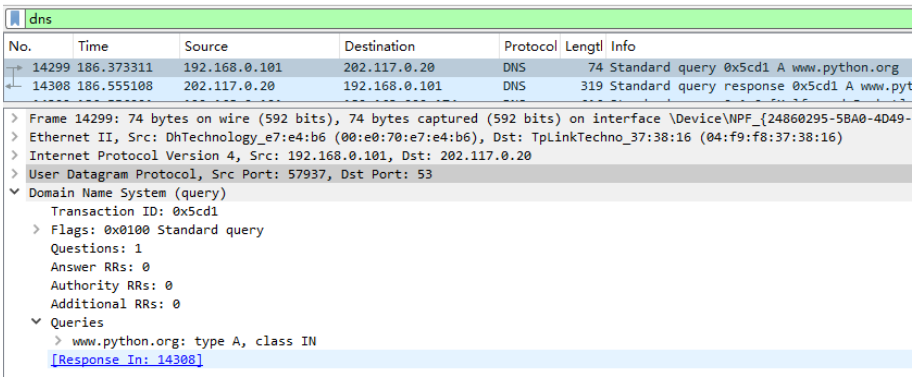
TXT 记录：可任意填写，可为空。一般做一些验证记录时会使用此项，如：做 SPF（反垃圾邮件）记录；

SOA 记录：SOA 叫做起始授权机构记录，NS 用于标识多台域名解析服务器，SOA 记录用于在众多 NS 记录中那一台是主服务器；

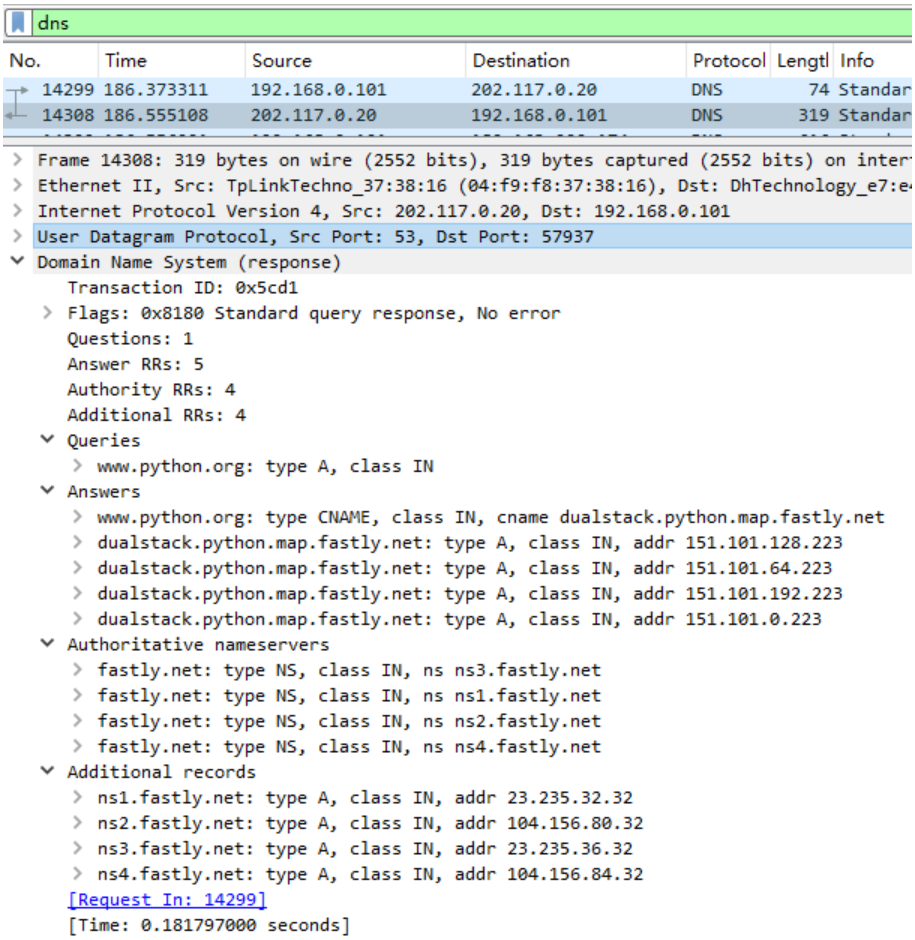
显性 URL 转发记录：将域名指向一个 http(s)协议地址，访问域名时，自动跳转至目标地址。

隐性 URL 转发记录：将域名指向一个 http(s)协议地址，访问域名时，自动跳转至目标地址，隐性转发会隐藏真实的目标地址。

query：



response：



步骤 3：**指定服务器的 DNS 域名解析**。在命令窗口执行命令：

执行“ipconfig /displaydns”观察本地缓存；

执行“ipconfig /flushdns”清除本地缓存。

在计算机终端上运行 Wireshark 截获报文，在命令窗口执行指定 DNS 服务器解析域名命令（如 nslookup www.github.com **223.6.6.6**），解析完毕后停止报文截获，观察分析 DNS 查询、回复报文并按报告填表。

2.5.2 HTTP 协议分析

步骤 4：无（DNS、HTTP）缓存情况下的**网站页面**访问速度测试。

在计算机终端上运行 Wireshark 开始截获报文。清空 DNS 和 HTTP 浏览器的缓存：

浏览器缓存的清除以 Chrome 浏览器为例，地址栏中输入 chrome://settings/，找到高级选项中的“隐私设置和安全性”，清除浏览数据。

在 cmd 命令行执行“ipconfig /flushdns”清除本地 DNS 缓存（或者在 PowerShell 用 Clear-DnsClientCache/Get-DnsClientCache 命令清除/查看 DNS 缓存）。

在浏览器中访问 3 个网址，分别是国内、国外和南半球（指非洲或南美洲）网址。

注意：必须指定 http，目前很多网站默认是 https，不容易分析其内容；如果想对 HTTPS 的协议交互内容进行 wireshark 解析，可浏览器的 F12 调试功能，或问 AI。

国内网址：如 http://www.xjtu.edu.cn（http://www.hit.edu.cn 或 http://www.people.com.cn/）。国内、国外和南半球支持 HTTP 的网址请参考《补充材料-支持 HTTP 的网站》。

网页加载完成之后，停止报文截获并保存数据，分析截获的报文，填写下表。

表 2-1 无缓存时不同网站访问情况统计表

访问网站	国内	国外	南半球
域名			
IP 地址			
RTT			
DNS 次数			
第 1 个 DNS 请求的应答时间(ms)			
页面的加载时间(ms)			
请求对象数			
TCP 连接数			

下面简单解释一下表格中某些项目的含义及获取方法。这些项目都跟浏览器下载和加载网页有关系。一个网页中往往有多个内嵌对象，在加载网页时，这些内嵌对象都会一起下载。有些内嵌对象也许并不存储在这个服务器上。如果某内嵌对象对应的其它网址，则可能会触发新的 DNS 请求，并且需要 TCP 连接到这个服务器上。即便是多个对象都在一个服务器上，浏览器也可能会建立新的 TCP 连接去下载它们，因为这样往往用户感受到的延迟更小。

RTT：按下载主页面的连接建时间计算。注意跟实际的物理距离结合判断其合理性。

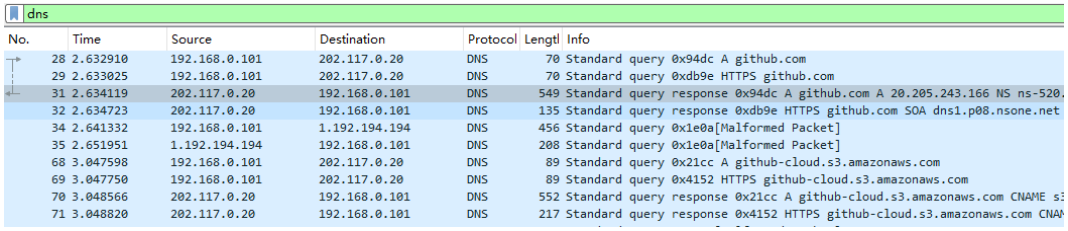
请求对象数：按网页中内嵌对象的个数来计算。用 Wireshark 找也可以，但很难统计准确。

DNS 次数：除了主页面的域名外，如果某内嵌对象的网址不是主服务器，并且也不在 DNS 缓存中，则会触发新的 DNS 解析。

可以编程来计算请求对象数和 DNS 次数。

第 1 个 DNS 请求的应答时间(ms)：**第 1 个 DNS 请求**指的是主页面对应的 DNS 请求，本地 DNS 服务器收到了这个请求，往往会启动递归的 DNS 解析。这个应答时间长短一般由这个递归 DNS 解析的时间来定，如果这个域名在本地 DNS 服务器乃至顶级服务器中都没有缓存，则触发到二级域乃至三级 DNS 服务器的查询，国外特别是南半球的二级域和对应子域 DNS 服务器的延迟非常大（几百 ms），所以解析可能会花比较长的时间。Wireshark 中找到对应的两个 DNS 分组，计算其时间差即可。也可以编程来计算它。

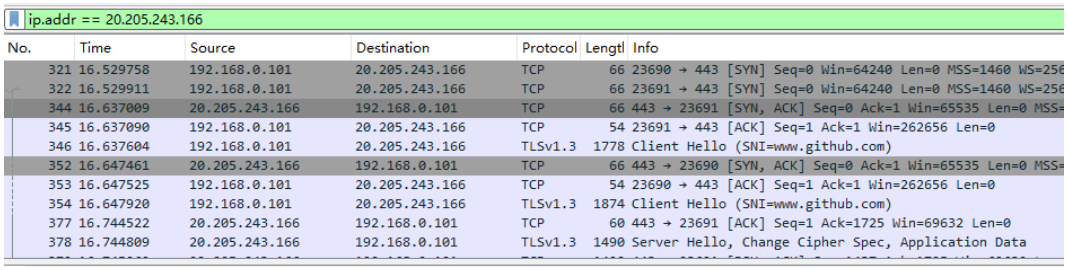
如下图中，分组 28 对应的应答报文号是 31，计算其时间差为 2.634119-2.632910，大约为 1ms。



dns

No.	Time	Source	Destination	Protocol	Length	Info
28	2.632910	192.168.0.101	202.117.0.20	DNS	70	Standard query 0x94dc A github.com
29	2.633025	192.168.0.101	202.117.0.20	DNS	70	Standard query 0xdb9e HTTPS github.com
31	2.634119	202.117.0.20	192.168.0.101	DNS	549	Standard query response 0x94dc A github.com A 20.205.243.166 NS ns-520.
32	2.634723	202.117.0.20	192.168.0.101	DNS	135	Standard query response 0xdb9e HTTPS github.com SOA dns1.p08.nsone.net
34	2.641332	192.168.0.101	1.192.194.194	DNS	456	Standard query 0x1e0a[Malformed Packet]
35	2.651951	1.192.194.194	192.168.0.101	DNS	208	Standard query 0x1e0a[Malformed Packet]
68	3.047598	192.168.0.101	202.117.0.20	DNS	89	Standard query 0x21cc A github-cloud.s3.amazonaws.com
69	3.047750	192.168.0.101	202.117.0.20	DNS	89	Standard query 0x4152 HTTPS github-cloud.s3.amazonaws.com
70	3.048566	202.117.0.20	192.168.0.101	DNS	552	Standard query response 0x21cc A github-cloud.s3.amazonaws.com CNAME s3
71	3.048820	202.117.0.20	192.168.0.101	DNS	217	Standard query response 0x4152 HTTPS github-cloud.s3.amazonaws.com CNAM

RTT 的计算举例，通过 IP 地址过滤数据包，TCP 连接建立时分组 322 和 344 的时间差为 16.637009-16.529911，大约为 107ms。

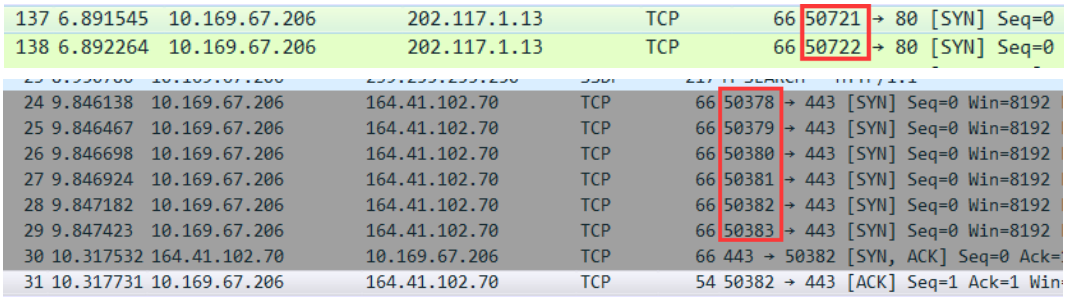


ip.addr == 20.205.243.166

No.	Time	Source	Destination	Protocol	Length	Info
321	16.529758	192.168.0.101	20.205.243.166	TCP	66	23690 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256
322	16.529911	192.168.0.101	20.205.243.166	TCP	66	23691 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256
344	16.637009	20.205.243.166	192.168.0.101	TCP	66	443 → 23691 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=
345	16.637090	192.168.0.101	20.205.243.166	TCP	54	23691 → 443 [ACK] Seq=1 Ack=1 Win=262656 Len=0
346	16.637604	192.168.0.101	20.205.243.166	TLSv1.3	1778	Client Hello (SNI=www.github.com)
352	16.647461	20.205.243.166	192.168.0.101	TCP	66	443 → 23690 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=
353	16.647525	192.168.0.101	20.205.243.166	TCP	54	23690 → 443 [ACK] Seq=1 Ack=1 Win=262656 Len=0
354	16.647920	192.168.0.101	20.205.243.166	TLSv1.3	1874	Client Hello (SNI=www.github.com)
377	16.744522	20.205.243.166	192.168.0.101	TCP	60	443 → 23691 [ACK] Seq=1 Ack=1725 Win=69632 Len=0
378	16.744809	20.205.243.166	192.168.0.101	TLSv1.3	1490	Server Hello, Change Cipher Spec, Application Data

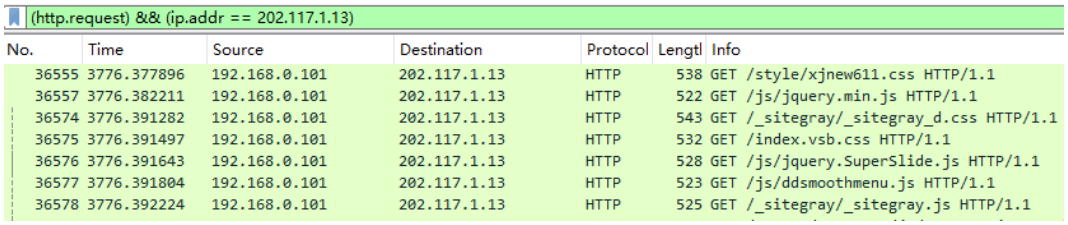
页面的加载时间(ms)：加载时间是网页从开始请求到完整展现所花的时间。一个网页的加载时间包括其内嵌对象的（下载）加载数据，有些内嵌对象也许并不存储在这个服务器上。所以这里网页的**加载时间**，如果从 Wireshark 来观察，就是下载该网页涉及到的所有的 DNS、HTTP 报文中**最后一个**报文与第一个报文间隔时间。通过 Wireshark 还不能直接显示出加载时间，需要人工分析。

TCP 连接数：下载主页面和所有内嵌对象时实际的 TCP 连接的数量。注意：如果内嵌对象的网址与主页面的网址不同，则一定会建立 TCP 连接；如果相同，也可能会建立新的 TCP 连接去下载，因为多个连接下载速度快。如果所有内嵌对象都跟主页面是同一个网址，则 Wireshark 统计 TCP 连接数比较容易（如下图）。否则，需要细心观察，或编程辅助（统计有哪些对象来自其它网址）。



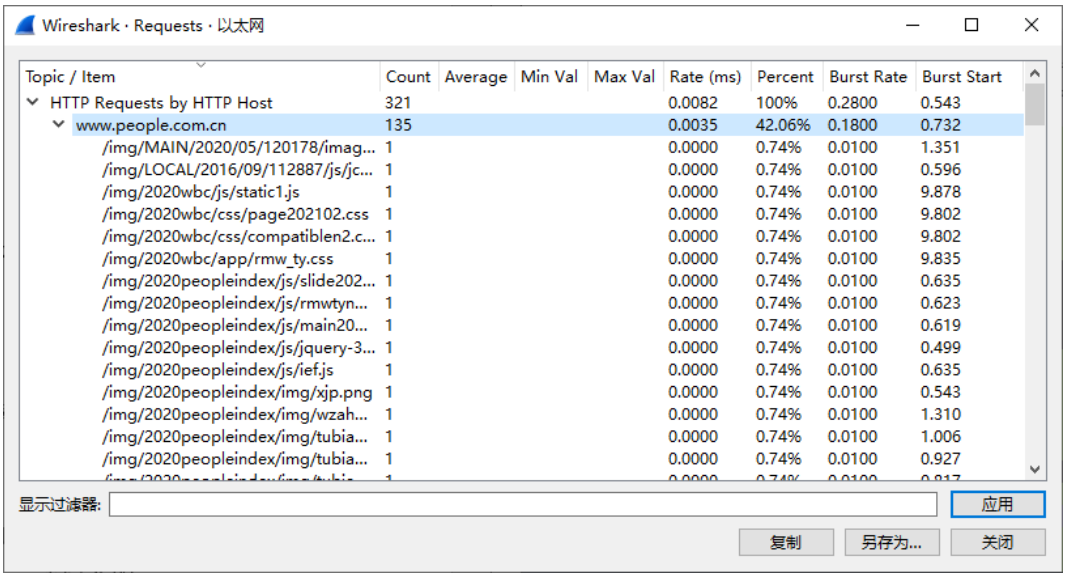
137	6.891545	10.169.67.206	202.117.1.13	TCP	66	50721 → 80 [SYN] Seq=0
138	6.892264	10.169.67.206	202.117.1.13	TCP	66	50722 → 80 [SYN] Seq=0
24	9.846138	10.169.67.206	164.41.102.70	TCP	66	50378 → 443 [SYN] Seq=0 Win=8192
25	9.846467	10.169.67.206	164.41.102.70	TCP	66	50379 → 443 [SYN] Seq=0 Win=8192
26	9.846698	10.169.67.206	164.41.102.70	TCP	66	50380 → 443 [SYN] Seq=0 Win=8192
27	9.846924	10.169.67.206	164.41.102.70	TCP	66	50381 → 443 [SYN] Seq=0 Win=8192
28	9.847182	10.169.67.206	164.41.102.70	TCP	66	50382 → 443 [SYN] Seq=0 Win=8192
29	9.847423	10.169.67.206	164.41.102.70	TCP	66	50383 → 443 [SYN] Seq=0 Win=8192
30	10.317532	164.41.102.70	10.169.67.206	TCP	66	443 → 50382 [SYN, ACK] Seq=0 Ack=
31	10.317731	10.169.67.206	164.41.102.70	TCP	54	50382 → 443 [ACK] Seq=1 Ack=1 Win

用 Wireshark 找**请求对象**也可以，统计未必准确。**请求对象**如：HTML，CSS，JS，图片等，可以通过设置过滤条件筛选，或者通过菜单栏的“统计->HTTP->请求”查看：



(http.request) && (ip.addr == 202.117.1.13)

No.	Time	Source	Destination	Protocol	Lengtl	Info
36555	3776.377896	192.168.0.101	202.117.1.13	HTTP	538	GET /style/xjnew611.css HTTP/1.1
36557	3776.382211	192.168.0.101	202.117.1.13	HTTP	522	GET /js/jquery.min.js HTTP/1.1
36574	3776.391282	192.168.0.101	202.117.1.13	HTTP	543	GET /_sitegray/_sitegray_d.css HTTP/1.1
36575	3776.391497	192.168.0.101	202.117.1.13	HTTP	532	GET /index.vsb.css HTTP/1.1
36576	3776.391643	192.168.0.101	202.117.1.13	HTTP	528	GET /js/jquery.SuperSlide.js HTTP/1.1
36577	3776.391804	192.168.0.101	202.117.1.13	HTTP	523	GET /js/ddsmoothmenu.js HTTP/1.1
36578	3776.392224	192.168.0.101	202.117.1.13	HTTP	525	GET /_sitegray/_sitegray.js HTTP/1.1



Wireshark · Requests · 以太网

Topic / Item	Count	Average	Min Val	Max Val	Rate (ms)	Percent	Burst Rate	Burst Start
HTTP Requests by HTTP Host	321				0.0082	100%	0.2800	0.543
www.people.com.cn	135				0.0035	42.06%	0.1800	0.732
/img/MAIN/2020/05/120178/imag...	1				0.0000	0.74%	0.0100	1.351
/img/LOCAL/2016/09/112887/js/jc...	1				0.0000	0.74%	0.0100	0.596
/img/2020wbc/js/static1.js	1				0.0000	0.74%	0.0100	9.878
/img/2020wbc/css/page202102.css	1				0.0000	0.74%	0.0100	9.802
/img/2020wbc/css/compatiblen2.c...	1				0.0000	0.74%	0.0100	9.802
/img/2020wbc/app/rmw_ty.css	1				0.0000	0.74%	0.0100	9.835
/img/2020peopleindex/js/slide202...	1				0.0000	0.74%	0.0100	0.635
/img/2020peopleindex/js/rmwtyn...	1				0.0000	0.74%	0.0100	0.623
/img/2020peopleindex/js/main20...	1				0.0000	0.74%	0.0100	0.619
/img/2020peopleindex/js/jquery-3...	1				0.0000	0.74%	0.0100	0.499
/img/2020peopleindex/js/ief.js	1				0.0000	0.74%	0.0100	0.635
/img/2020peopleindex/img/xjp.png	1				0.0000	0.74%	0.0100	0.543
/img/2020peopleindex/img/wzah...	1				0.0000	0.74%	0.0100	1.310
/img/2020peopleindex/img/tubia...	1				0.0000	0.74%	0.0100	1.006
/img/2020peopleindex/img/tubia...	1				0.0000	0.74%	0.0100	0.927

显示过滤器: 应用

复制 另存为... 关闭

步骤 5：有（DNS、HTTP）缓存情况下的**网站页面**访问速度测试。

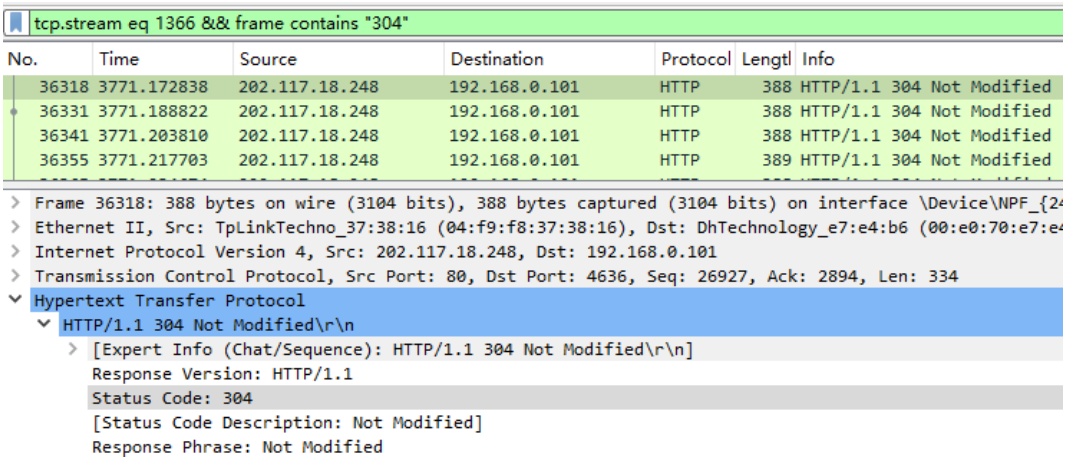
按上一节中的步再次执行一遍，但不清除 DNS、HTTP 缓存，填写下表信息。根据所学知识分析缓存技术会给页面访问带来哪些方面的改善，这些改善在上面的步骤 4、5 中有没有实际发生？请结合两个表格的数据说一说。

带缓存情况下不同网站访问情况统计表

访问网站	国内	国外	南半球
域名			
IP 地址			
RTT			
DNS 次数			
页面的加载时间(ms)			
请求对象数			
TCP 连接数			

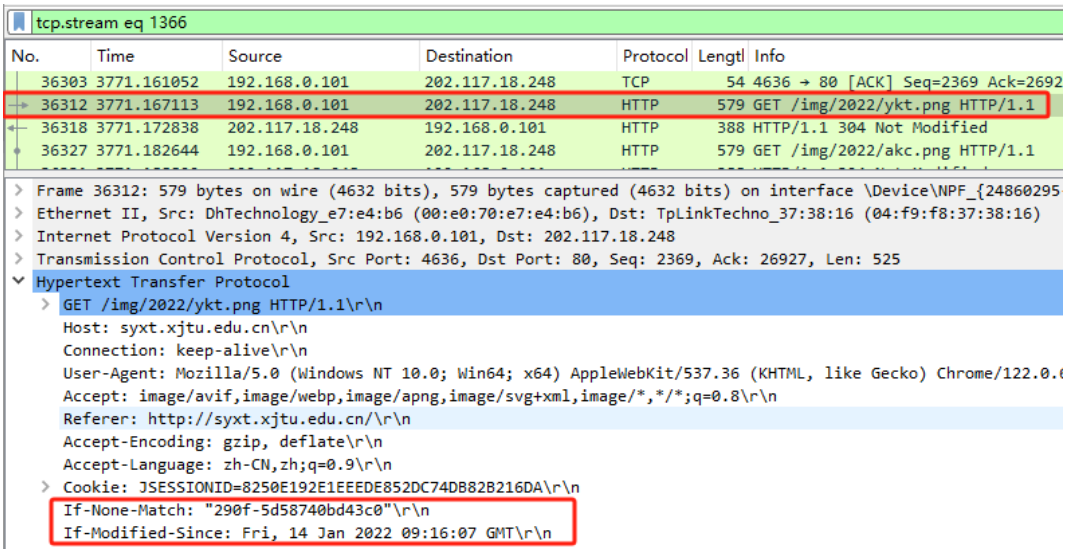
步骤 6：分析 HTTP 的缓存工作原理。

找到 HTTP 应答包中**状态码为 304** 的数据包（数据包过多时可通过过滤条件 frame contains "304"快速查找）。



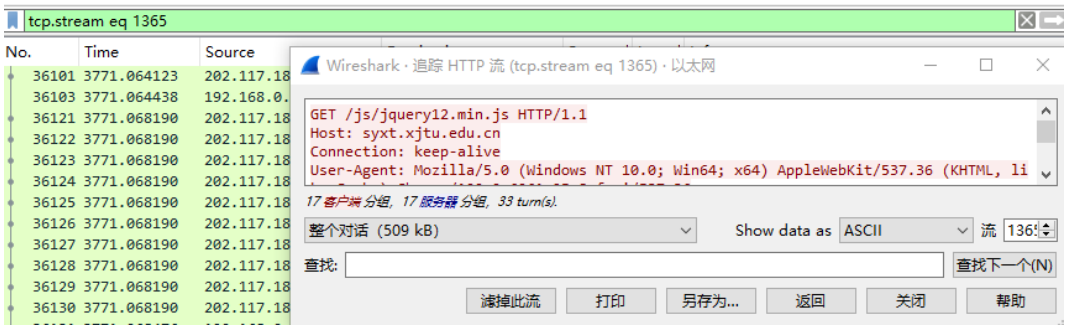
如果找不到，可以再次打开 http://www.xjtu.edu.cn 等 HTTP 网站抓包，多次刷新、重复打开页面。

找到 304 应答包对应的**请求包**，分析缓存工作原理和带来的好处。



步骤 7：HTTP 流的跟踪。

选中一条 HTTP 报文，点击右键，选择“追踪流->HTTP Stream”。



关闭统计窗口，在主界面观察数据流，结合数据包分析 HTTP 的请求和应答，以及与 TCP 协议的配合使用。

tcp.stream eq 1366

No.	Time	Source	Destination	Protocol	Lengtl	Info
36068	3771.045279	192.168.0.101	202.117.18.248	TCP	66	4636 → 80 [SYN] Seq=0 Win=64240 Len
36077	3771.053154	202.117.18.248	192.168.0.101	TCP	66	80 → 4636 [SYN, ACK] Seq=0 Ack=1 Wi
36084	3771.053745	192.168.0.101	202.117.18.248	TCP	54	4636 → 80 [ACK] Seq=1 Ack=1 Win=263
36088	3771.054015	192.168.0.101	202.117.18.248	HTTP	542	GET /css/video-js.min.css HTTP/1.1
36102	3771.064123	202.117.18.248	192.168.0.101	TCP	60	80 → 4636 [ACK] Seq=1 Ack=489 Win=3
36110	3771.064504	202.117.18.248	192.168.0.101	TCP	1494	80 → 4636 [ACK] Seq=1 Ack=489 Win=3
36111	3771.064504	202.117.18.248	192.168.0.101	TCP	1494	80 → 4636 [ACK] Seq=1441 Ack=489 Wi
36112	3771.064504	202.117.18.248	192.168.0.101	TCP	1494	80 → 4636 [ACK] Seq=2881 Ack=489 Wi
36113	3771.064504	202.117.18.248	192.168.0.101	TCP	1494	80 → 4636 [ACK] Seq=4321 Ack=489 Wi
36114	3771.064843	192.168.0.101	202.117.18.248	TCP	54	4636 → 80 [ACK] Seq=489 Ack=5761 Wi
36115	3771.064909	202.117.18.248	192.168.0.101	TCP	1494	80 → 4636 [ACK] Seq=5761 Ack=489 Wi
36116	3771.064909	202.117.18.248	192.168.0.101	TCP	1494	80 → 4636 [ACK] Seq=7201 Ack=489 Wi
36117	3771.064909	202.117.18.248	192.168.0.101	HTTP	1483	HTTP/1.1 200 OK (text/css)

步骤 8：使用 ncat 工具访问某 HTTP 网站。

Nmap(Network Mapper)是一款功能强大的开源网络扫描工具，其包含的工具 Ncat 可以用于网络连接、调试和数据传输。Nmap 软件解压后，在 cmd 命令行进入目录，就可以执行命令，也可以将软件目录添加至系统环境变量。

前面学习了 HTTP 协议，也用 Wireshark 分析了典型的 HTTP 请求和应答。现在使用 ncat 来**模拟**浏览器来跟服务器进行 HTTP 交互。在命令窗口执行“ncat -C 域名/ip 地址 端口（80）”连接 HTTP 服务器（如 www.xjtu.edu.cn），输入 HTTP 请求（可参考之前捕获的数据包中的请求），请求头至少有 Host 和 User-Agent，其中 **User-Agent 后跟上自己的英文名字**。查看服务器的应答。

例：

ncat -C **xxx.xxx.xxx.xxx** 80　**（xxx.xxx.xxx.xxx 是具体的域名或 IP 地址）**

GET / HTTP/1.1

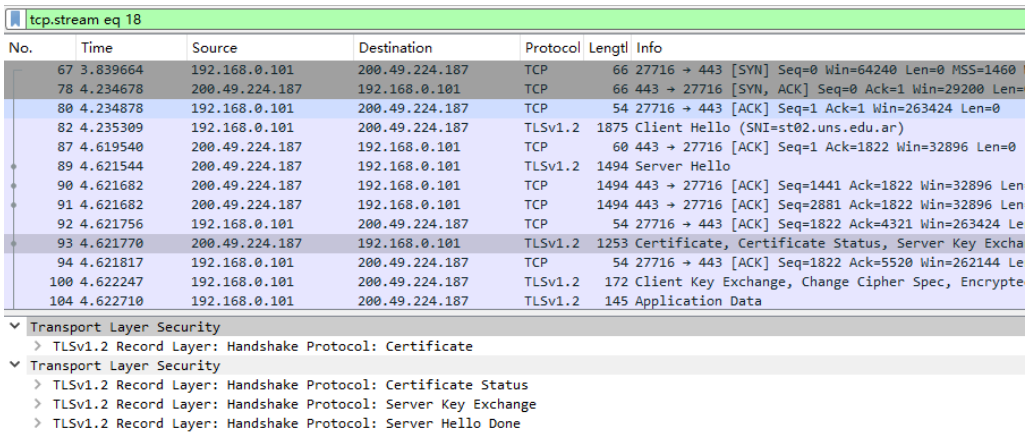
Host: **xxx.xxx.xxx.xxx（xxx.xxx.xxx.xxx 是具体的域名。）**

User-Agent: Zhang San （**跟上自己的英文名字。**然后敲两次回车，应答状态码 200 表示请求成功）

按报告要求把 ncat 命令及其输出用文字贴出来。

步骤 9：HTTPS 协议分析。

访问 HTTPS 网站时，因数据是加密传输无法观察内容，可结合原理图与抓到的数据包**分析 TLS 加密传输的建立过程、传输端口**等。



tcp.stream eq 18

No.	Time	Source	Destination	Protocol	Length	Info
67	3.839664	192.168.0.101	200.49.224.187	TCP	66	27716 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
78	4.234678	200.49.224.187	192.168.0.101	TCP	66	443 → 27716 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=
80	4.234878	192.168.0.101	200.49.224.187	TCP	54	27716 → 443 [ACK] Seq=1 Ack=1 Win=263424 Len=0
82	4.235309	192.168.0.101	200.49.224.187	TLSv1.2	1875	Client Hello (SNI=st02.uns.edu.ar)
87	4.619540	200.49.224.187	192.168.0.101	TCP	60	443 → 27716 [ACK] Seq=1 Ack=1822 Win=32896 Len=0
89	4.621544	200.49.224.187	192.168.0.101	TLSv1.2	1494	Server Hello
90	4.621682	200.49.224.187	192.168.0.101	TCP	1494	443 → 27716 [ACK] Seq=1441 Ack=1822 Win=32896 Len
91	4.621682	200.49.224.187	192.168.0.101	TCP	1494	443 → 27716 [ACK] Seq=2881 Ack=1822 Win=32896 Len
92	4.621756	192.168.0.101	200.49.224.187	TCP	54	27716 → 443 [ACK] Seq=1822 Ack=4321 Win=263424 Le
93	4.621770	200.49.224.187	192.168.0.101	TLSv1.2	1253	Certificate, Certificate Status, Server Key Excha
94	4.621817	192.168.0.101	200.49.224.187	TCP	54	27716 → 443 [ACK] Seq=1822 Ack=5520 Win=262144 Le
100	4.622247	192.168.0.101	200.49.224.187	TLSv1.2	172	Client Key Exchange, Change Cipher Spec, Encrypte
104	4.622710	192.168.0.101	200.49.224.187	TLSv1.2	145	Application Data

Transport Layer Security
- TLSv1.2 Record Layer: Handshake Protocol: Certificate

Transport Layer Security
- TLSv1.2 Record Layer: Handshake Protocol: Certificate Status
- TLSv1.2 Record Layer: Handshake Protocol: Server Key Exchange
- TLSv1.2 Record Layer: Handshake Protocol: Server Hello Done

2.5.3 编写 HTTP 客户端程序

步骤 10：编写 HTTP 客户端程序代码

（1）编写一个简单的 HTTP 客户端程序。建议使用 Python 的 socket 模块，但不能使用 Python 的高级 HTTP 模块（如 requests、urllib.request、http.client 等）。服务器的名字通过命令行参数（Python 中的 sys.argv，C 的 argv）给出。

（2）运行 TCP 客户端程序进行连接测试（如连接到某网站，获取并打印主页）。并记录 DNS 和 TCP 连接及获得 HTTP 完整应答的时间。

输入 HTTP 请求时，请求协议为 HTTP/1.1，除了请求行以外，请求头至少有 Host 和 User-Agent，其中 **User-Agent 后跟上自己的英文名字**。

2.6 进阶实验

步骤 11：改进 HTTP 客户端程序

改进步骤 10 中编写的 HTTP 客户端程序，能够完整下载一个页面，即除 HTML 页面外，还要下载所有的内嵌对象。但不是下载所有的链接。运行并记录结果，包括所有的 DNS 和 TCP 连接及获得 HTTP 完整应答的时间。